Computabilidad en Blockchain

Bitcoin - Ethereum

Autómatas y Gramáticas · UNLaM · DIIT

JTP: Ing. Weber Federico

Disclaimer

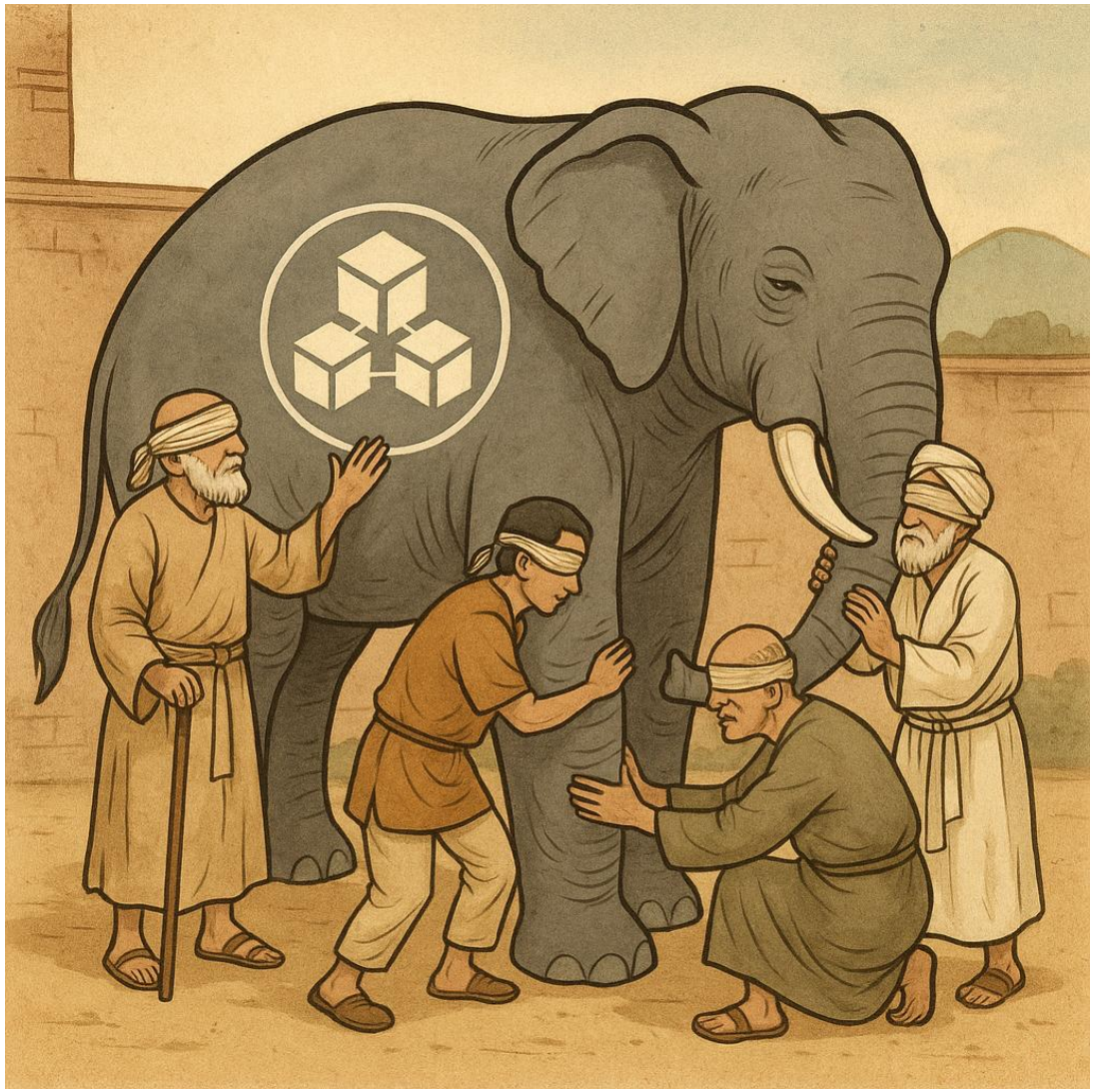
El movimiento

El casino

El producto

La tecnología

Hoy no vemos «todo Bitcoin, ni todo Ethereum». Nos ponemos las gafas concretas de esta materia.



Bitcoin

0. Se conoce a «Bitcoin» por dos cosas distintas.
   Bitcoin (mayúscula) la red y el protocolo: las computadoras y las reglas que siguen.
   bitcoin (minúscula) la moneda que circula en esa red. Unidad: BTC.

1. Bitcoin es una red p2p, sin servidor central. Los nodos corren un software que sigue las mismas reglas para todos. El programa más usado es Bitcoin Core (es una implementación).

2. La red mantiene un ledger público con todas las transacciones, visibles para cualquiera.

3. Para mover monedas se usa criptografía asimétrica: una clave privada firma, cualquiera verifica con la pública.

4. **Las condiciones para bloquear y desbloquear monedas son un programa chico, escrito en Script, que corre en una máquina virtual dentro de cada nodo.**

5. Resultado: dinero digital sin intermediarios. Nadie autoriza; el código de cada nodo verifica.

Qué hay adentro de una transacción

Entradas: qué monedas se gastan, y la prueba de que se puede (unlocking script).

Salidas: cuánto va a cada destino, y la condición para gastarlo después (locking script).

La condición no es una regla fija del sistema, es un programa y depende de lo que “especifique” quien recibe.

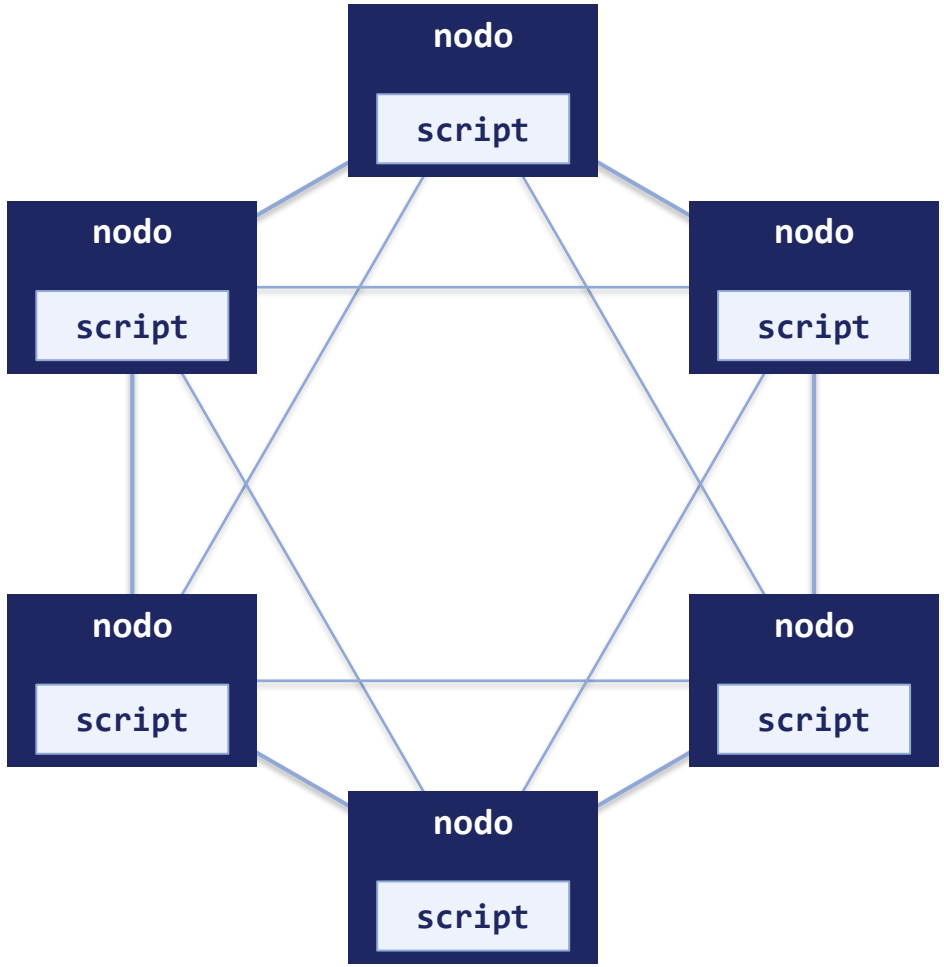


El script , en cada nodo

Cada nodo ejecuta el script de cada transacción, por su cuenta y lo valida.

No hay un servidor central que tome decisiones de ningún tipo. Todos los nodos son iguales.

¿qué clase de máquina es la que ejecuta ese programa?



Decide sí o no

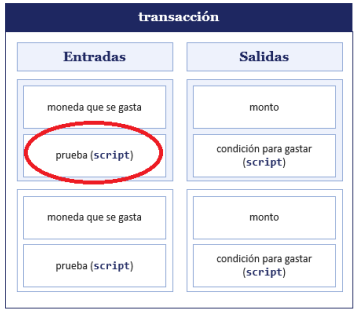
Un script no lee archivos, no abre sockets, no mira el reloj.

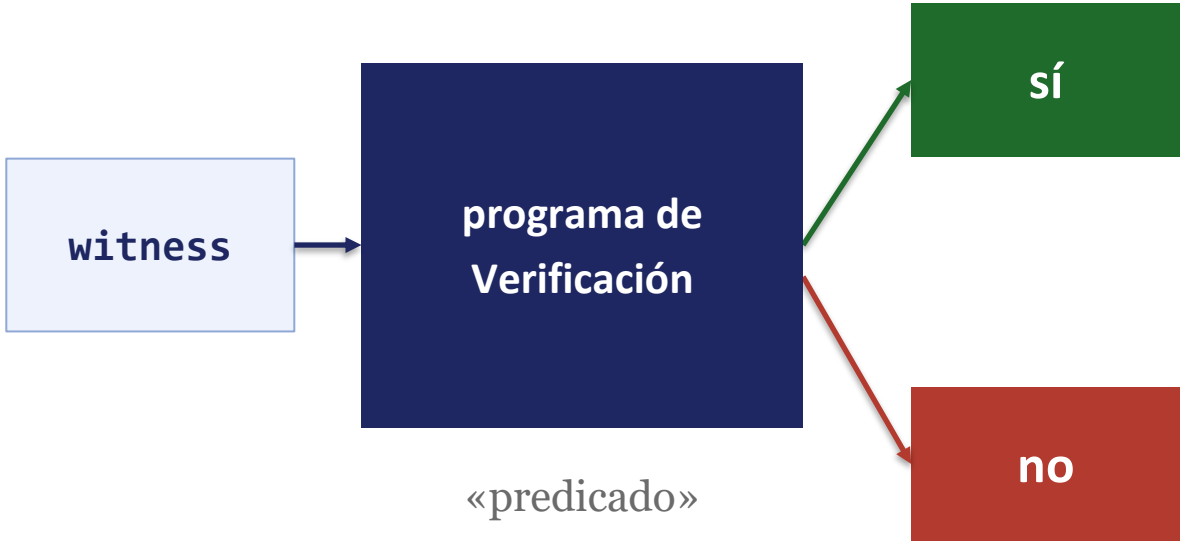
Decide una sola cosa: ¿esta transacción tiene derecho a gastar estas monedas?

La condición la escribió quien recibió las monedas (salida de una transacción anterior).

La prueba (witness) la trae quien las gasta (entrada de la transacción nueva).

Si la prueba satisface la condición, la entrada vale.
Predicado: una función que devuelve sí o no.





Si un script pudiera colgarse, una transacción colgaría todos los nodos del mundo.

Desde 2009

Desde el 3 de enero de 2009

uptime > 99,98 %

Dos incidentes: 2010 (**CVE-2010-5139**) y 2013 (**CVE-2013-3220**)



https://bitcoinuptime.org/

Un script es una secuencia de datos y opcodes

Un script es una secuencia de elementos. Se lee de izquierda a derecha, de a uno.

Cada elemento es un dato (un número, una firma, una clave) o un opcode (una operación).

Un dato se apila. Un opcode saca lo que necesita de arriba de la pila, calcula, y deja el resultado arriba.

La única memoria es la pila. A una máquina así se le dice máquina de pila.

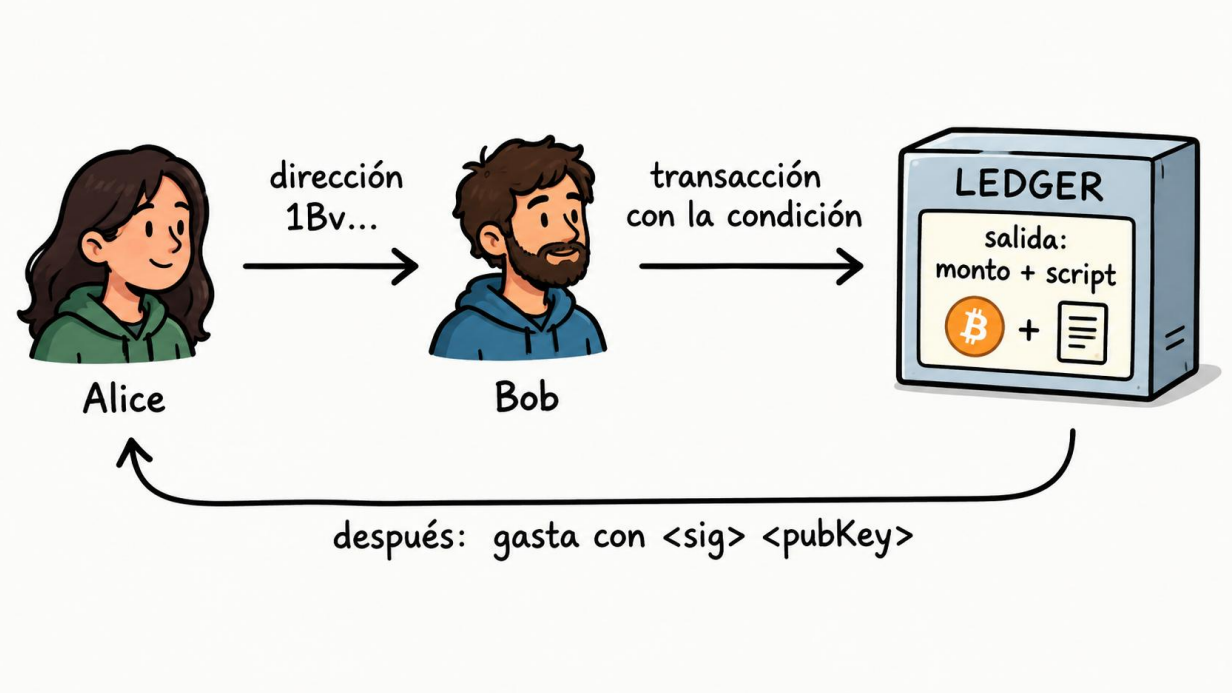
Ejemplo: **`2 3 OP_ADD 5 OP_EQUAL`** («¿2 + 3 es igual a 5?»)



Cinco pasos y termina. Queda 01 arriba de la pila: «sí». Si quedara 00, o nada: «no».

Un script es una secuencia de datos y opcodes

Con Bitcoin Script alcanza para la tarea fundamental del sistema: pagar.

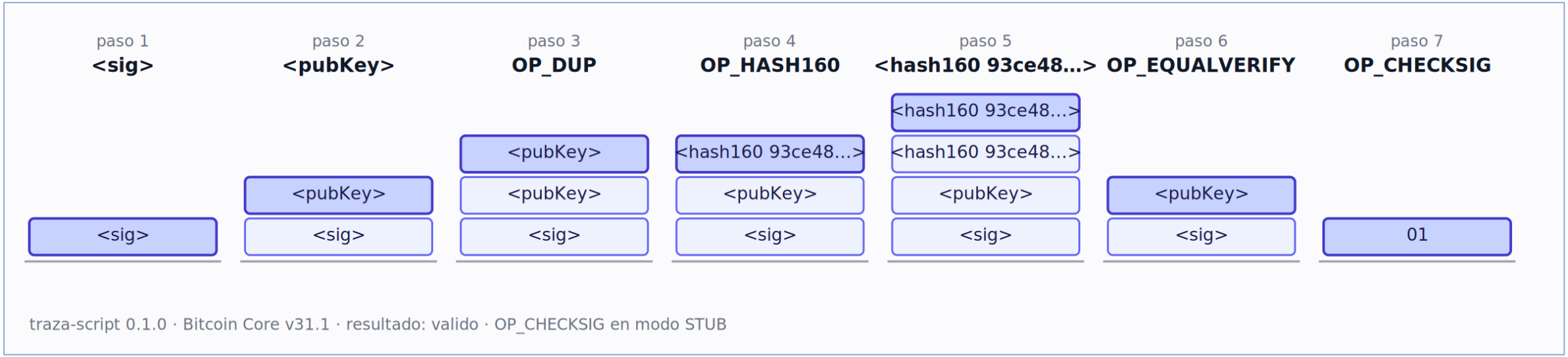


Bob le tiene que pagar a Alice.
Alice le da una dirección: codifica la condición que ella impone para gastar las monedas que va a recibir.
Bob arma la transacción con esa condición, y demuestra que puede gastar las monedas que pone.
Mañana Alice será Bob: para gastar lo que recibió, va a tener que demostrar que cumple su propia condición.
Cada entrada de cada transacción pasa por esto.

La traza de una transacción P2PKH

prueba (unlocking script, de la entrada):
`<sig> <pubKey>`

condición (locking script, de la salida):
`OP_DUP OP_HASH160 <hash160> OP_EQUALVERIFY OP_CHECKSIG`



Paso	Instrucción	Pila (de abajo a arriba)
1	<sig>	<sig>
2	<pubKey>	<sig> <pubKey>
3	OP_DUP	<sig> <pubKey> <pubKey>
4	OP_HASH160	<sig> <pubKey> <hash160>
5	<hash160>	<sig> <pubKey> <hash160> <hash160>
6	OP_EQUALVERIFY	<sig> <pubKey>
7	OP_CHECKSIG	01

Primero corre la prueba, después la condición, sobre la misma pila.

Cada opcode consume de arriba y deja su resultado arriba. Siete pasos y termina: 01 es «sí».

Características del Script de Bitcoin

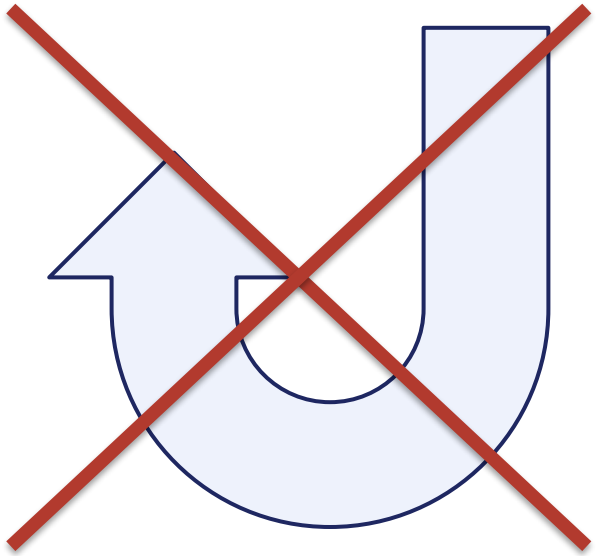
✗ Sin saltos hacia atrás: no hay GOTO, no hay bucles.

✗ Sin estado entre ejecuciones: cada script arranca de cero.

✗ Sin reloj, sin entrada/salida: no ve el mundo, solo la pila.

✓ Tiene condicionales: **`OP_IF`** … **`OP_ELSE`** … **`OP_ENDIF`**, incluso anidados. Puede bifurcar.

✓ Tiene aritmética, comparación, hash y verificación de firma.



ningún salto hacia atrás

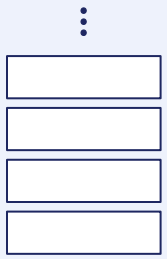
Características del Script de Bitcoin

MEMORIA (la pila)

1000

MAX_STACK_SIZE

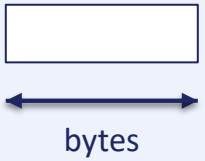
elementos en la pila, como máximo (contando la pila auxiliar)



520

MAX_SCRIPT_ELEMENT_SIZE

bytes por elemento, como máximo (una firma ocupa ~72; un hash, 20 o 32)



PROGRAMA (el script)

10000

MAX_SCRIPT_SIZE

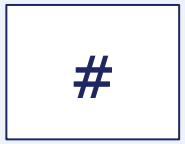
bytes de programa, como máximo



201

MAX_OPS_PER_SCRIPT

operaciones por script, como máximo (sin contar los push de datos)



Memoria acotada y programa acotado. Se sabe cuánto puede ocupar y cuánto puede trabajar un script antes de ejecutarlo.

Características del Script de Bitcoin

¿Es un lenguaje de programación Turing completo?

Un sistema (lenguaje de programación, máquina, formalismo) es Turing completo si tiene la misma potencia computacional que una Máquina de Turing.

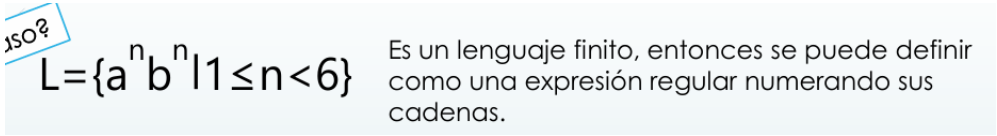
Condicionales (decisiones: if/else)	✓
Iteración o recursión (bucles o llamadas recursivas	✗
Memoria ilimitada (en teoría) .	✗

¿Dónde lo ubicamos nosotros?

Memoria acotada ⇒ cantidad finita de estados posibles (astronómica, pero finita) ⇒ autómata finito ⇒ Tipo 3.
El modelo más simple alcanza.

¿Por qué autómata finito y no autómata de pila?

- Un autómata finito se llama así «porque la cantidad de estados es limitada». Puede contar, si hay un máximo:



  Lo que no puede es contar sin máximo: $a^n b^n$ para todo n. Eso lo resuelve el autómata de pila.

- ¿Y la pila de Script?
  Tiene máximo: 1000 elementos de hasta 520 bytes.
  La profundidad de los OP_IF también: la fija el tamaño del programa (10000 bytes).
  Todo lo que Script «cuenta», lo cuenta hasta un número fijo. Eso lo hacen los estados.

- El máximo no está en el hardware, sino que está en la definición.
  En C o Java la memoria es ilimitada «en teoría». El lenguaje no pone tope, lo pone la computadora.
  En Script el tope es una regla de consenso: un script que necesite 1001 elementos no es grande, es inválido, en cualquier máquina.

¿no podría ser mas limitado, cierto?

2010: quince opcodes menos

Encima de los límites de diseño, en 2010 Script quedó aún más limitado. Con un **`if`**.

```
457  if (opcode == OP_CAT ||
458      opcode == OP_SUBSTR ||
459      opcode == OP_LEFT ||
460      opcode == OP_RIGHT ||
461      opcode == OP_INVERT ||
462      opcode == OP_AND ||
463      opcode == OP_OR ||
464      opcode == OP_XOR ||
465      opcode == OP_2MUL ||
466      opcode == OP_2DIV ||
467      opcode == OP_MUL ||
468      opcode == OP_DIV ||
469      opcode == OP_MOD ||
470      opcode == OP_LSHIFT ||
471      opcode == OP_RSHIFT)
472      return set_error(serror, SCRIPT_ERR_DISABLED_OPCODE); // Disabled opcodes (CVE-2010-5137).
```

Quince opcodes apagados con un if, entre los cuales multiplicar, dividir, concatenar, operar bit a bit.

Lo que había detrás del if ya no existe en el código. Script es simple adrede.

`Bitcoin Core v31.1, src/script/interpreter.cpp:457-472`

¿Cuánto cuesta multiplicar dos enteros de 32 bits?

Atari 2600 (1977)

Algunas instrucciones



Bitcoin Script (hoy)

3.565 bytes de script

no hay **OP_MUL**: se multiplica sumando
casi todo **OP_ADD** y **OP_DROP**

2009: una instrucción, OP_MUL.
2010: se amputan los OPCODES.
2010-2021: sin OP_MUL y con 201 operaciones por script, no entra.
Desde Taproot (2021) se puede: 3.565 bytes, sumando.

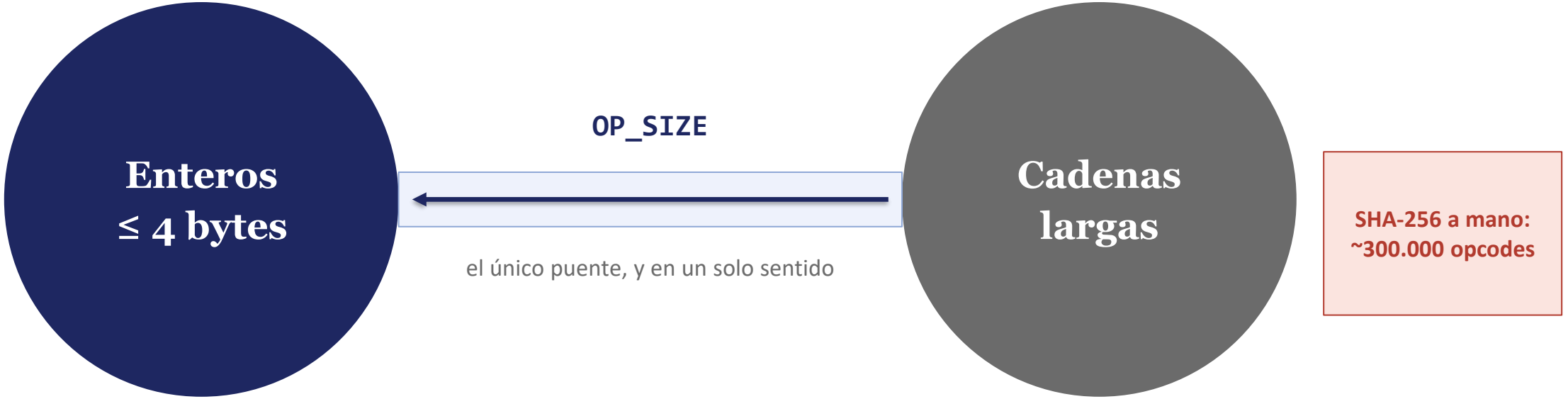
Más limitaciones:

Enteros (≤ 4 bytes): se suman, se comparan. **OP_ADD**, **OP_EQUAL**…

Cadenas largas (20-72 bytes): hashes, firmas. **OP_HASH160**, **OP_CHECKSIG**…

Ningún opcode lleva de un universo al otro. Salvo uno: **OP_SIZE** (de una cadena, devuelve su largo como entero).

Podés hashear. Pero con el hash no podés hacer nada más que compararlo.

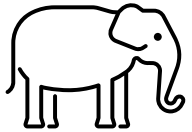


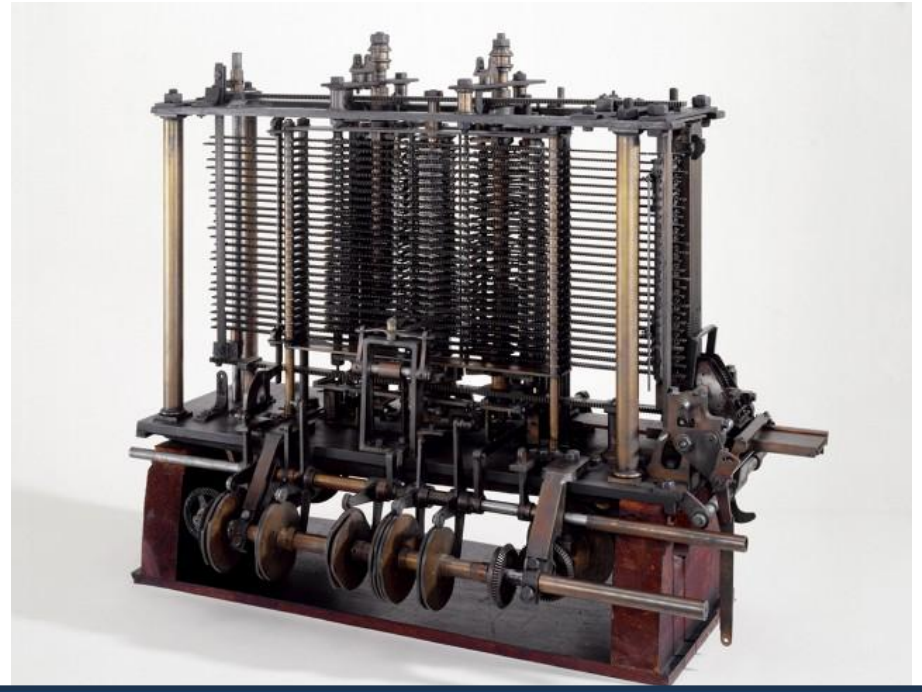
Script de bitcoin



En poder computacional, esto que corre en cada nodo es una calculadora que suma, resta y tiene un botón interesante: **`OP_CHECKSIG`**.

Todo esto nos lleva a pensar que a nadie le interesaría intentar construir algo sobre bitcoin.





Si Babbage hubiera terminado la Maquina Analítica diseñada en 1837, habría sido más potente que Bitcoin Script.

Características de bitcoin



- **Inmutable**
- **Permisionless**
- **Incesurable**
- **Verificable**

Atractivo, pero limitado

Inmutable

Permisionless

Resistente a la censura

Verificable por cualquiera

→

Un lugar atractivo para construir una capa de actividad financiera global.

pero

Poder de cómputo:
una calculadora con un botón copado.

Se quisiera hacer, encima de esta base, más de lo que la base puede computar.

¿Y por qué no cambiar Bitcoin directamente?

- No es solo un problema técnico: es político y social.
- Cambiar las reglas de consenso requiere que miles de nodos independientes adopten el cambio. Nadie puede obligarlos.
- En general, el consenso no se logra.

2010	quince opcodes apagados con un if. En 2026, siguen apagados.
2021	Taproot (BIP-341): el último cambio de consenso activado. Años de discusión.
2023	OP_CAT (BIP-347): propuesta formal. No activada.

Dos caminos (los que vemos hoy nosotros)

Otra infraestructura, desde cero

Ethereum

Ethereum (2015): una red nueva, con un lenguaje Turing completo.

Se hace todo adentro: el cómputo corre en cada nodo.

lo vemos más adelante

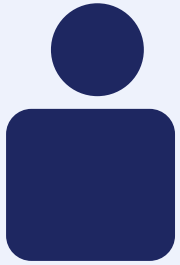
Bitcoin, se construye sobre lo que existe

Computación optimista (o disputable).

Computación optimista

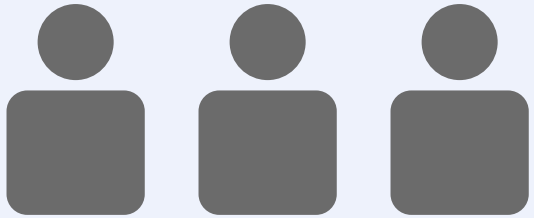
La idea principal es quitar la lógica compleja de Bitcoin (capa base) y ejecutarla en otra capa. Bitcoin se usa para **garantizar que el resultado sea correcto.**

Para esa capa nueva aparecen dos jugadores nuevos con su infraestructura.



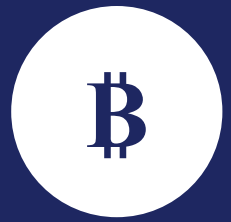
Operador

Ejecuta el programa afuera y afirma el resultado.



Verificadores

Vuelven a ejecutar por su cuenta y vigilan. Alcanza con uno honesto.



Bitcoin: el juez

No ejecuta el programa. Interviene solo si hay disputa.

Antes de empezar: ponerse de acuerdo OFFCHAIN

Operador y verificadores acuerdan tres cosas antes de que nada corra:

1 El programa

Lo que quieren ejecutar: un puente a otra red, una DAO, lo que sea.

Mucho más complejo que cualquier script.

2 El entorno

Una CPU emulada.
Del programa tienen el binario.

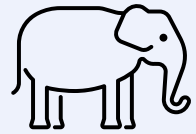
Un binario se sigue instrucción por instrucción de esa CPU: paso 1, paso 2, …, paso N.



3 Las reglas

Cuánto deposita cada parte como garantía.

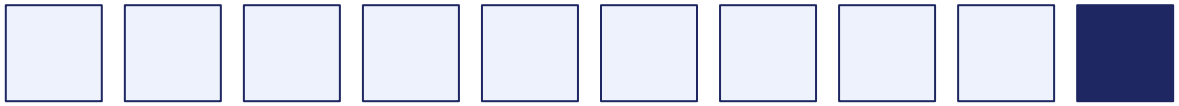
Cuánto dura cada plazo para reclamar o responder.



Todo esto se fija antes. Después nadie puede cambiar las reglas a mitad del juego.

El operador ejecuta y publica

OFF - CHAIN (la máquina del operador)



paso 1 … paso N: el programa entero, millones de instrucciones

La traza completa queda acá. No se publica.

Corre en milisegundos. Nadie más la vio todavía.

publica

ON-CHAIN (Bitcoin)

las entradas
con qué datos corrió el programa

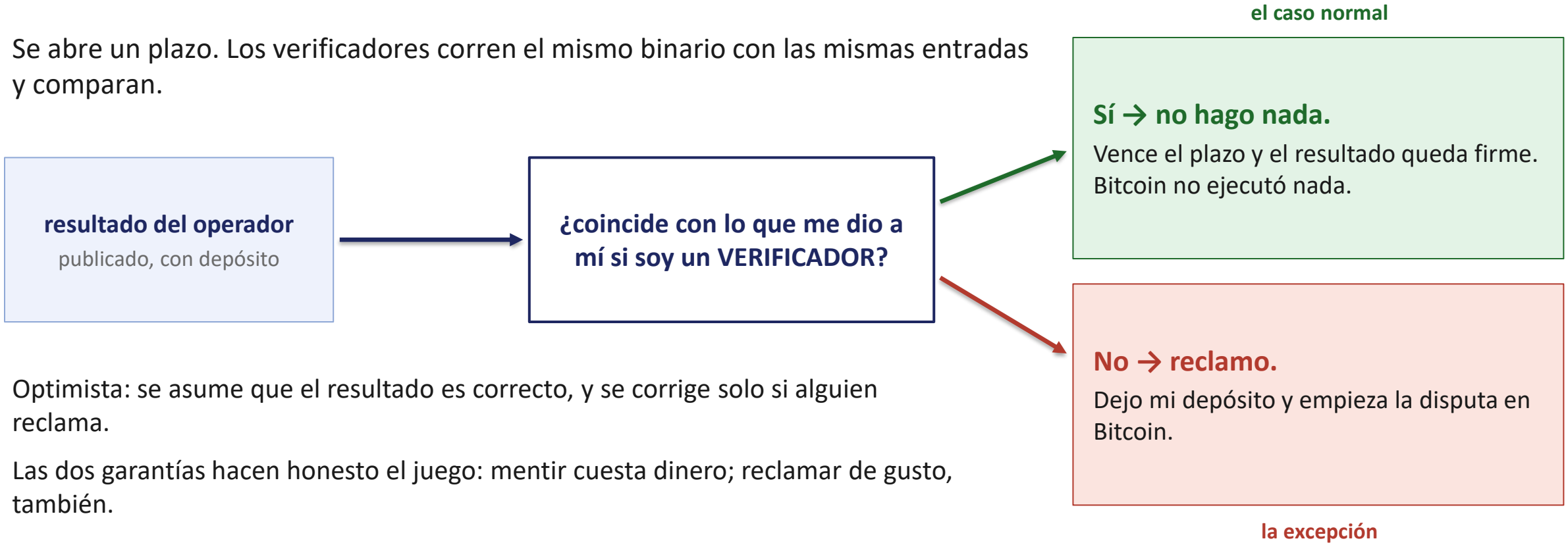
el resultado
lo que el operador afirma que dio

su depósito
la garantía de que no miente

Tres cosas chicas en Bitcoin. Nadie verificó nada todavía: es una afirmación con una garantía por detrás.

Por qué se llama «optimista»

Se abre un plazo. Los verificadores corren el mismo binario con las mismas entradas y comparan.



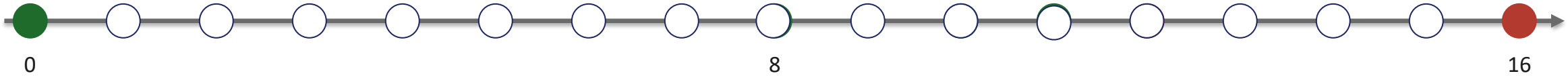
Optimista: se asume que el resultado es correcto, y se corrige solo si alguien reclama.

Las dos garantías hacen honesto el juego: mentir cuesta dinero; reclamar de gusto, también.

Si nadie reclama, la base no trabajó. Por eso el costo normal es casi nulo.

La disputa: búsqueda binaria sobre los pasos

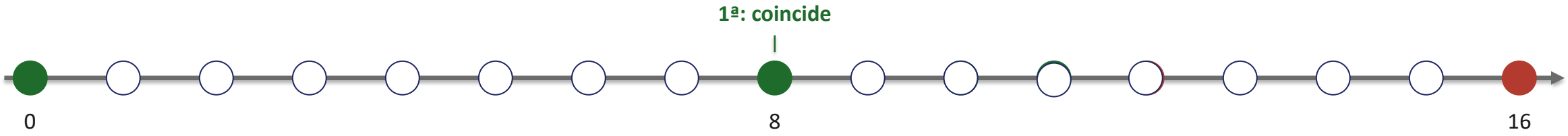
Coinciden en el paso 0 (mismas entradas) y difieren en el último (el resultado). ¿Dónde empezó la diferencia?



1 El verificador pide un paso del medio.

La disputa: búsqueda binaria sobre los pasos

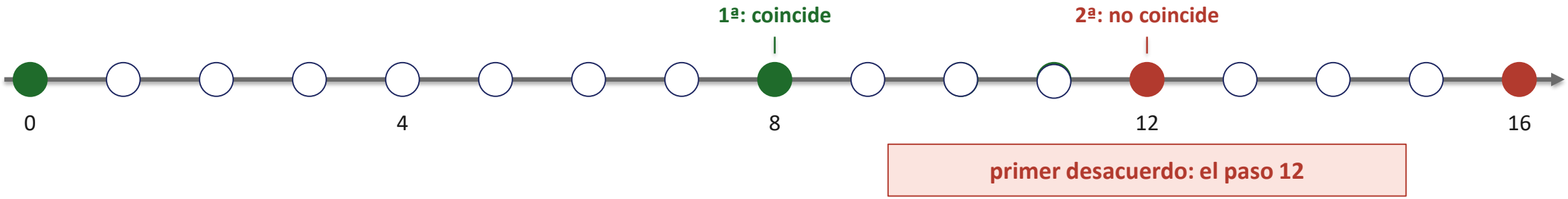
Coinciden en el paso 0 (mismas entradas) y difieren en el último (el resultado). ¿Dónde empezó la diferencia?



1. El verificador pide un paso del medio.
2. El operador responde: muestra el estado de la máquina en ese paso («una foto»).
3. El verificador compara con su ejecución, dice si coincide, y elige la mitad donde seguir.

La disputa: búsqueda binaria sobre los pasos

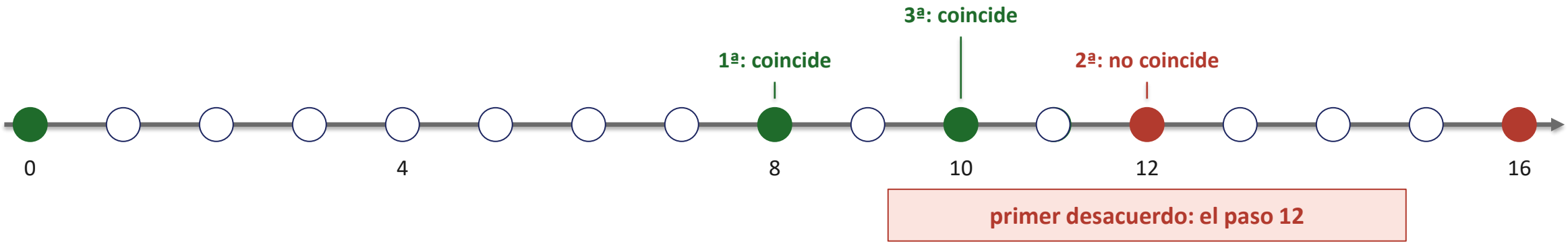
Coinciden en el paso 0 (mismas entradas) y difieren en el último (el resultado). ¿Dónde empezó la diferencia?



1. El verificador pide un paso del medio.
2. El operador responde: muestra el estado de la máquina en ese paso («una foto»).
3. El verificador compara con su ejecución, dice si coincide, y elige la mitad donde seguir.

La disputa: búsqueda binaria sobre los pasos

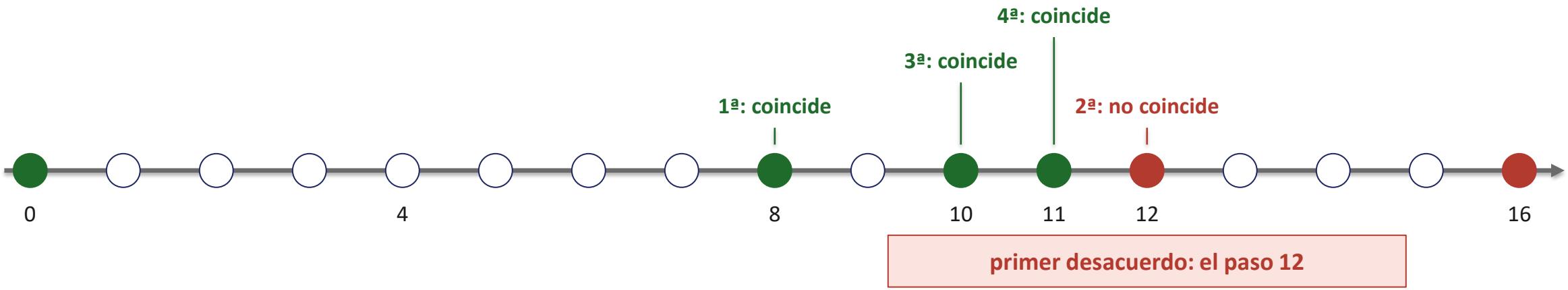
Coinciden en el paso 0 (mismas entradas) y difieren en el último (el resultado). ¿Dónde empezó la diferencia?



1. El verificador pide un paso del medio.
2. El operador responde: muestra el estado de la máquina en ese paso («una foto»).
3. El verificador compara con su ejecución, dice si coincide, y elige la mitad donde seguir.

La disputa: búsqueda binaria sobre los pasos

Coinciden en el paso 0 (mismas entradas) y difieren en el último (el resultado). ¿Dónde empezó la diferencia?



1. El verificador pide un paso del medio.
2. El operador responde: muestra el estado de la máquina en ese paso («una foto»).
3. El verificador compara con su ejecución, dice si coincide, y elige la mitad donde seguir.

Cada pregunta parte el problema a la mitad.

16 pasos → 4 preguntas

1.000.000 de pasos → 20 preguntas

2^{32} pasos → 32 preguntas

Pocas rondas para aislar un único paso. Ese paso es lo único que va a tocar Bitcoin.

Ese único paso lo verifica Bitcoin

Una instrucción de la CPU sí entra en un script de Bitcoin (1 a 5 KB).

El script toma el estado anterior, aplica esa única instrucción y compara con lo que afirmó el operador.

esquema, no código real, psuedocodigo

```
1  <estado 11>           // los dos coinciden acá
2  <instrucción 12>      // la que dice el binario
3  ejecutar un paso      // una instrucción: Script puede
4  <estado 12 afirmado>  // lo que publicó el operador
5  OP_EQUAL              // ¿coincide?
```

Da falso

El operador mintió. Pierde su depósito; el verificador lo cobra.

Da verdadero

El verificador se equivocó. Pierde su depósito; el operador lo cobra.

Y si alguien deja de responder, pasado el plazo pierde.

Bitcoin no ejecutó el programa. Ejecutó un paso, y con eso alcanzó para castigar al que mintió.

Bitcoin, el juez



1. Bitcoin nunca hace el cómputo. En el caso normal, no hace nada.
2. En el peor caso, ejecuta un paso. Uno.
3. Solo se pronuncia si alguien levanta la mano.
4. Alcanza con un verificador honesto mirando.

Lo que corre on-chain sigue siendo un script chico que decide sí o no. Lo que cambió es el protocolo alrededor.

Esto tiene nombre

El paradigma se llama computación optimista o disputable. En Bitcoin tiene dos nombres propios:

2023 BitVM

Robin Linus

Demostró que el juez puede ser Bitcoin Script, sin cambiar una línea de consenso.

Su versión original verificaba compuertas lógicas, no instrucciones de CPU.

2024 BitVMX

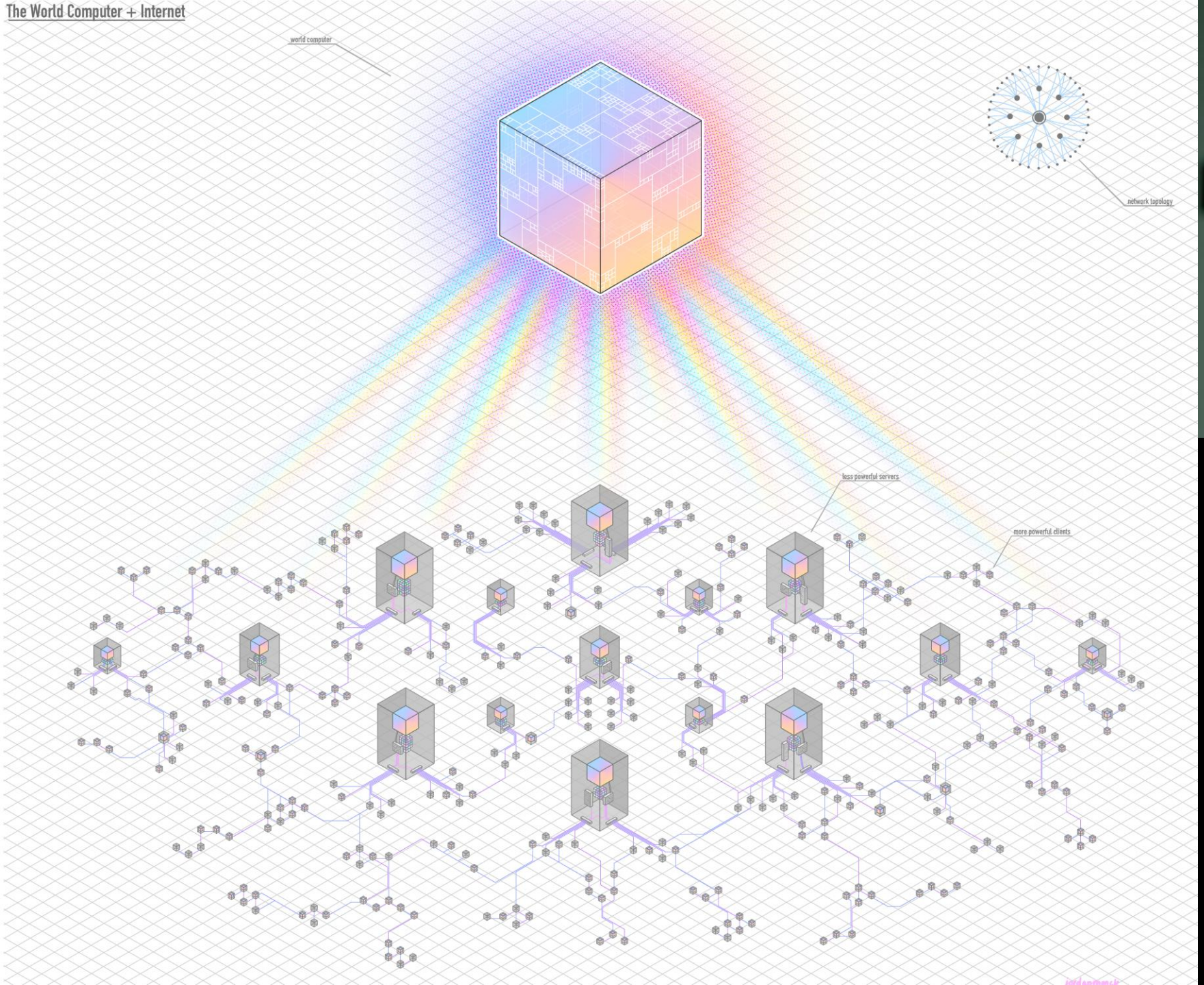
Sergio Lerner y equipo (Rootstock Labs / Fairgate)

La versión con una CPU RISC-V emulada: exactamente lo que acabamos de describir.

Un paso de CPU = un script de 1 a 5 KB.

La idea es más vieja que Bitcoin: viene de la teoría de la verificación interactiva (años 80 y 90) y se probó primero en Ethereum (TrueBit, 2017; optimistic rollups).

Ninguna de las dos cambió Bitcoin. Construyeron sobre lo que existe.





Ethereum “the world computer”

Mas poder en la capa base

Antes de los modelos tipo BitVM se pensó lo más obvio: si la base tiene poco poder computacional, hagamos otra base con más.

Ethereum (2015): una máquina virtual Turing-completa:
Saltos hacia atrás (`JUMP`, `JUMPI`), memoria, storage persistente.

Hay varios lenguajes de programación Turing completos para este tipo de maquina. El mas conocido se llama Solidity.

Contrapartida: las propiedades de Bitcoin no se copian. No están en el código ni en la tecnología. Son propiedades que emergieron de un sistema que funcionó y del paso del tiempo.

Solidity (usado en blockchain)

`JUMP / JUMPI`

memoria

storage

Tipo o

Problema de la parada

Turing-completo → problema de la parada:

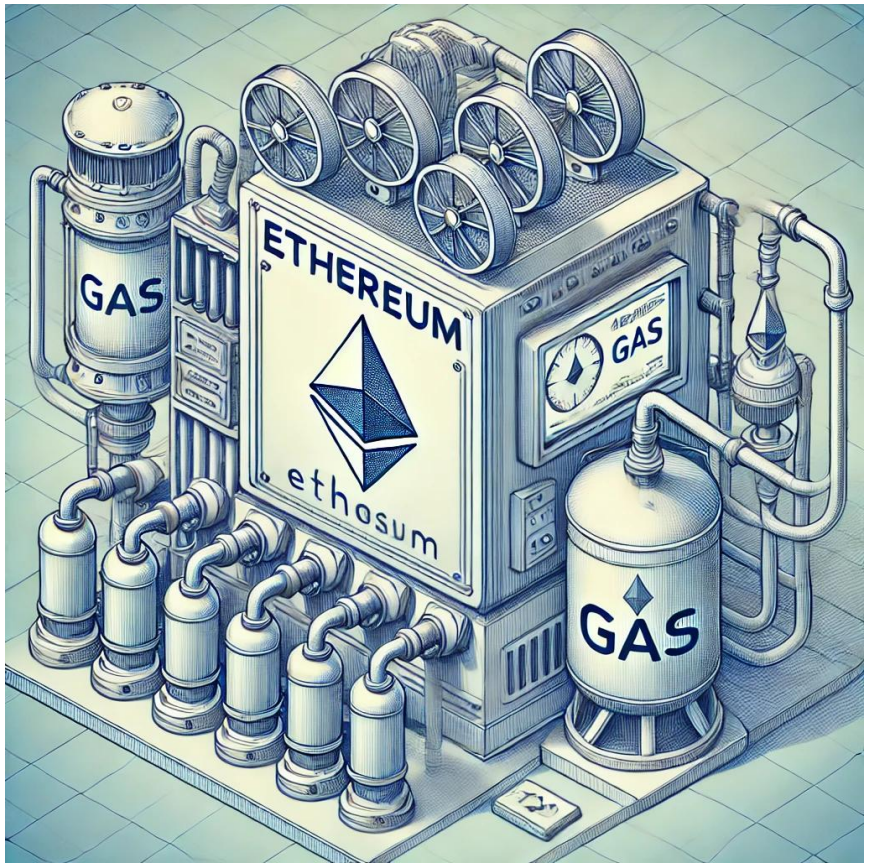
un nodo no puede decidir de antemano si un contrato termina.

Solución de ingeniería → gas.

Cada instrucción cuesta;

la transacción trae un tope;

si se agota el gas, se revierte la transacción pero la tarifa se paga igual.



fee = (basefee + priorityfee) * gasused

https://www.evm.codes/

La velocidad de escape funcional



Vitalik Buterin (2019).

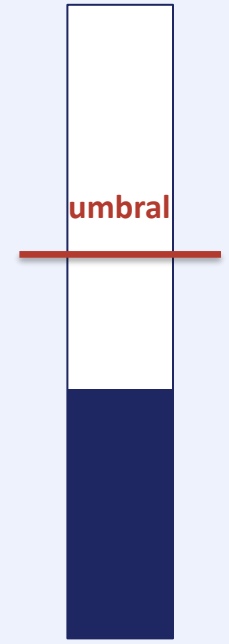
Velocidad de escape funcional: lo mínimo que tiene que poder hacer la capa base para que todo lo demás se construya encima sin volver a cambiarla.

Si la base llega, nunca más hay que tocarla.
Si no llega, las capas de arriba tienen que volver a confiar en alguien.

Lo que pide del lenguaje: poder verificar cualquier cosa. No hace falta que sea Turing completo.

> *«It's okay to restrict to programs without loops, or programs which are guaranteed to terminate in a specific number of steps.»* Vitalik Buterin, 2019

Alcanza con programas sin bucles, o que terminan en una cantidad fija de pasos. Eso es lo que acabamos de clasificar como autómata finito.



poder de la base

Arriba del umbral:
todo lo demás se construye encima; la base no se toca más.

Abajo del umbral:
las capas de arriba vuelven a depender de confiar en alguien.

¿Script llega? Solo, no. Con el protocolo de disputa, sí.

Dos modelos diferentes

Bitcoin	Ethereum
La base restringe el lenguaje: Tipo 3 por diseño.	La base permite todo: Tipo 0.
La terminación es estructural; el costo se conoce antes.	El gas cobra por paso.
Lo que falta se construye encima, por protocolo: BitVM disputa.	No hay nada encima de Turing completo.*

*existen otras capas encima de Ethereum para resolver diferentes temas, pero no la computabilidad.

El mismo problema: ejecutar código ajeno en cada nodo del mundo sin que nadie pueda colgarlo. La misma pregunta: ¿cuánto poder necesita la base?

¿Qué tipo de lenguaje piensan que necesita la capa base de un sistema financiero global?

Gracias

Autómatas y Gramáticas · UNLaM · DIIT

JTP: Ing. Weber Federico